

How Invariant is Code Generation to Graph Serialization?

Measuring and Localizing the Fragility

Ram Kedem - 206844144, Yehonatan Barel - 208646422, Ido Azoulay - 212710958, Dolev Abudi - 323096099

TL;DR. We test whether solving graph problems with LLM-generated code returns the same answer when the same graph is serialized in equivalent ways, and localize at which step any fragility enters.

1 Background & Motivation

Setting. Consuming a graph $G = (V, E)$ directly is inherently difficult for an LLM: in practice, every graph task requires a *serialization* $g : G \mapsto \text{text}$ [2]. A single graph admits infinitely many equivalent serializations (arbitrary node labels, arbitrary edge order, edge list vs. adjacency list, plain text vs. JSON vs. natural language). Graph neural networks are permutation-invariant, an LLM operating on token sequences carries no such guarantee.

Interaction modes & research question. Given a serialized graph, an LLM can either answer directly, reading the text and answering in-context, or generate code: write a program (e.g., in NetworkX) that rebuilds the graph and computes the answer, which is then executed, its output taken as the answer. Code generation was recently found to be the strongest of these modes [3], yet its robustness to the serialization choice is unknown.

Our research question: **does solving a graph problem using code generation return the same answer under equivalent serializations of the same graph, and if not, at which step does the fragility enter?**

What is known.

- (i) Direct prompting is fragile. Fatemi et al. [2] showed the encoding choice alone shifts accuracy by 4.8 - 61.8% on basic graph tasks (GraphQA). Herbst et al. [5] confirmed this systematically: across $>1.5\text{M}$ inferences on the Erdős suite [4], decomposing serializations into node labeling, edge encoding, and syntax, they found reindexing, edge reordering, and format changes alter outputs on the same graph.
- (ii) Code is the strongest mode. Finkelshtein et al. [3] compared prompting, tool-use, and code generation across 14 datasets and models from Llama to GPT-5, finding code generation strongest overall. For example, on a synthetic shortest-path task it achieved zero error at all path lengths while other modes collapsed beyond 2 hops.

The gap.

	Direct answering	Executed code
Accuracy	measured (weaker)	measured (strongest)
Invariance to serialization	measured (<i>fragile</i>)	never measured

Finkelshtein et al. [3] evaluate code generation across many graphs, tasks, and competing modes, but always under one fixed serialization. Whether the answer survives re-serializing the same graph is never asked. Herbst et al. [5] ask exactly that question, but only for direct answering: NetworkX/PyG code appears in their prompts as a textual format the model reads, never as a program that runs. The one work that runs LLM-written code under more than one encoding is CodeGraph [1], which reports

average accuracy per GraphQA encoder and finds a narrower spread under code than under CoT. This measurement leaves the gap open for three reasons. First, it is aggregate: two encoders can have identical average accuracy while the model’s answer flips on every individual graph, so invariance must be measured per instance. Second, the perturbation is uncontrolled: GraphQA’s encoders change node labels, phrasing, and syntax all at once, and never vary edge order, so an observed drop cannot be attributed to any single serialization axis. Third, it is undiagnosed: when code mode fails, nothing distinguishes a mistranscribed graph from wrong solution logic or a crash. Answering our research question therefore means composing the two lines: applying the controlled, single-axis serialization perturbations of Herbst et al. [5] to the executed-code modes of Finkelshtein et al. [3].

Why it matters. LLMs are increasingly used to reason over graph-structured data in high-impact domains. Yet, despite a surge of interest, the field lacks a principled understanding of the reliability of its best-practice interaction mode: reliability, not average accuracy, gates deployment, and a system whose answer flips under an arbitrary relabeling of the same input is unsafe. Our audit answers both halves of the research question: whether code generation is invariant to serialization, and, when it is not, at which step the fragility enters.

2 Proposed Approach

Core idea. Our audit has two goals: **measure** whether code generation is invariant to serialization, and, if it is not, **localize** the failure, which we sample at two steps: (a) transcription, where the LLM converts the serialized graph into code, and (b) solution logic, where it writes the code that computes the answer (a wrong result here may stem from a wrong algorithm choice or wrongly generated code, and we do not distinguish between the two). Since a graph can be prompted into the model in many ways, we do not pick the starting point arbitrarily: the canonical serialization of every graph is the best-performing encoding reported in prior work [2]. From this canonical form we derive provably equivalent variants by varying one serialization axis at a time, and test three interaction modes on all of them, yielding a factorial **invariance audit**: (perturbation axis) $\times$ (interaction mode).

Technical outline. We evaluate three interaction modes, each with a distinct role:

- M1. Direct** - single-turn answer from the serialized graph, the known-fragile baseline [5].
- M2. Code-1shot** - the model writes a full Python program from the serialized text, which we execute (the one-shot setup of CodeGraph [1]). It performs both steps (a) and (b) and carries the measurement goal.
- M3. Graph-as-Code** - the graph is provided as a programmatic data structure and the model writes code that solves the problem over that representation, without reading serialized text [3]. Step (a) is absent, so contrasting it with Code-1shot carries the localization goal.

The pipeline then runs as follows:

- (1) Sample graphs and tasks.
- (2) Generate the equivalent variants of each graph following Herbst et al. [5], each differing from the canonical form in exactly one axis: node relabeling, edge reordering, or syntax, etc.
- (3) Query every variant under all three modes with fixed prompts: Direct and Code-1shot receive the variant serialized in the prompt, while Graph-as-Code receives it as code representation.
- (4) Execute generated code.
- (5) Score answers against ground truth.
- (6) Automatically classify every code failure by the step it occurred in: a transcription error means the program built the wrong graph (we extract the graph object it constructed and compare its normalized edge set to the true one), a logic error means the graph is correct but the code computes the wrong answer, and an execution error means the program crashed or timed out.

Novelty. Our audit is the first to measure invariance of executed code at the instance level and under perturbations isolated to a single serialization axis, the first to decompose code-mode failures into transcription vs. solution logic, and the first to use a transcription-free control to separate the two. Together these make it a direct, falsifiable stress test of the field’s “use code” recommendation. Localizing

where the fragility enters is also the first step toward removing it, and if the audit’s findings allow, we will attempt a mitigation guided by the diagnosis.

3 Experimental Plan

Datasets. GraphQA [2]: basic tasks (edge existence, node degree, connected and disconnected nodes, cycle check) over small graphs. A subset of the Erdős suite [4] (50 graph-theoretic tasks) supplies harder tasks (shortest path, triangle counting) and is the suite on which Herbst et al. [5] measured direct-mode fragility. Optionally, NLGraph [6] adds eight further tasks up to maximum flow.

Models. A small open model (Qwen-7B) run on the Slurm cluster, and a 2 SOTA reasoning models accessed by API (DeepSeek-V4, Qwen 3.7-Max), at temperature 0 throughout.

Baselines / comparisons. (i) Across modes: Direct is the baseline mode and the reference for the invariance gap. (ii) Within each mode: the canonical serialization is the baseline for every perturbed variant. (iii) Between code modes: Graph-as-Code is the transcription-free control for Code-1shot, the comparison that carries localization. Externally, we compare our Code-1shot accuracies on GraphQA to CodeGraph’s published per-encoder results.

Metrics. Let $f_m(G_i^v)$ be the answer of mode m on variant v of graph G_i .

Performance. Accuracy against the ground truth $f^*(G_i)$: the fraction of pairs (G_i, v) with $f_m(G_i^v) = f^*(G_i)$, reported per (task, mode, axis).

Invariance. Measured per instance, following Herbst et al. [5]. For each graph G_i we check whether mode m returns the same answer on all of its equivalent variants: for discrete answers we record whether $f_m(G_i^1) = f_m(G_i^2) = \dots$, and for numeric answers we record the spread $\max_v f_m(G_i^v) - \min_v f_m(G_i^v)$, normalized so that 0 means identical answers. We then report the fraction of graphs answered identically and the mean spread. This ordering matters: CodeGraph [1] averages accuracy over graphs before comparing encoders, so a model that flips on every single graph can still look stable, whereas our per-graph check counts every flip.

Localization. Measured by the failure decomposition: among failed programs, the fractions whose extracted graph differs from the ground truth (transcription), whose graph is correct but whose answer is wrong (logic), or which crash (execution), per perturbation axis, together with the invariance gap between Code-1shot and Graph-as-Code. Success means a decisive verdict per task and axis: either code generation is invariant, or the failure decomposition says where it breaks.

References

- [1] Qiaolong Cai, Zhaowei Wang, Shizhe Diao, James Kwok, and Yangqiu Song. 2024. CodeGraph: Enhancing Graph Reasoning of LLMs with Code. arXiv:2408.13863.
- [2] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. Talk like a Graph: Encoding Graphs for Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.04560. Course paper list, Section 10.
- [3] Ben Finkelshtein, Silviu Cucerzan, Sujay Kumar Jauhar, and Ryen W. White. 2026. Actions Speak Louder than Prompts: A Large-Scale Study of LLMs for Graph Inference. In *International Conference on Learning Representations (ICLR)*. arXiv:2509.18487. Course paper list, Section 10.
- [4] Xiaojun Guo, Ang Li, Yifei Wang, Stefanie Jegelka, and Yisen Wang. 2025. G1: Teaching LLMs to Reason on Graphs with Reinforcement Learning (Erdős benchmark). In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2505.18499.
- [5] Daniel Herbst, Lea Karbevskaya, Divyanshu Kumar, Akanksha Ahuja, Fatemeh Gholamzadeh Nasrabadi, and Fabrizio Frasca. 2026. Lost in Serialization: Invariance and Generalization of LLM Graph Reasoners. In *AAAI Workshop on Graphs and more Complex Structures for Learning and Reasoning (GCLR)*. arXiv:2511.10234.
- [6] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can Language Models Solve Graph Problems in Natural Language?. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2305.10037.